MemPatch: Evidential Revision Scaffolds for Rapid Memory Integration in LLM Agents

Anonymous submission

Persistent-memory large language model (LLM) agents accumulate records to guide long-term retrieval and planning. However, when corrective or conflicting evidence arrives, agents struggle to revise the validity of already-admitted memory records, leading to the reuse of stale or contradicted facts. In this work, we address this challenge with two co-equal contributions. First, we introduce MemPatch-Bench, a diagnostic benchmark designed to evaluate post-admission memory validity revision. MemPatch-Bench requires agents to maintain record-level validity mappings (e.g., current, blocked, unresolved) under chronological evidence streams, scoring updates against hidden ground-truth states. Second, we propose MemPatch, a training-free, plug-and-play revision module. MemPatch separates opaque semantic proposals from verifiable state transitions by routing queries through a dual-path proposer interface—yielding global states and local action patches—and mediating updates via a deterministic Revision Guard and conservative reconciliation function. Our empirical evaluation of frozen open-weight backbones shows that standard prompting, retrieval-augmented generation (RAG), and memory baselines suffer severe integration failures. MemPatch significantly improves memory state consistency and eliminates stale-record reuse without parameter updates, establishing a verifiable and robust neuro-symbolic layer for agent memory.

Introduction

Large language model (LLM) agents are increasingly deployed in long-horizon environments where they must maintain state across extended temporal trajectories. Rather than relying solely on transient context windows, modern agent architectures employ persistent memory stores to accumulate experiences, document tool executions, and record intermediate planning states. These stored records actively guide subsequent system behaviors, serving as the knowledge base for downstream retrieval, planning, tool use, and database writes. In this persistent setting, a critical and under-addressed risk is record obsolescence—where a persistent record remains licensed for future reuse after corrective, conflicting, superseding, or policy-relevant evidence has arrived.

Retrieval alone does not solve this problem. For example, a calendar agent may persistently store that a user has a meeting scheduled on Tuesday. When subsequent email evidence reveals that the meeting has been moved to Friday, the agent must update its memory state. If the agent retrieves the new email and answers the immediate query correctly while leaving the contradicted Tuesday record active in its store, subsequent tasks may retrieve the stale Tuesday record, leading to scheduling conflicts. This silent failure permits immediate query success but leaves a contaminated state that propagates errors in future execution turns.

To isolate and evaluate this failure mode, we present MemPatch-Bench, a diagnostic benchmark specifically designed to evaluate post-admission validity revision in persistent agent memory. MemPatch-Bench requires agents to maintain record-level validity states under chronological evidence streams, scoring updates against hidden ground-truth labels. It evaluates agents not merely on their downstream answers, but on their ability to perform joint revision, scoring the concurrent correctness of task decisions, memory validity states, evidence citations, failure diagnoses, and answers.

To address this challenge without parameter updates, we propose MemPatch, a training-free, plug-and-play revision module that separates opaque semantic proposals from verifiable persistent-memory transitions. MemPatch does not expose the model's latent reasoning. It confines black-box inference to semantic proposal and makes every committed persistent-memory transition explicit, authorization-checked, and replayable. Specifically, MemPatch queries a frozen backbone model through a dual-path proposal interface to obtain global states and local typed patches. These proposals are compiled into structured actions and routed through a deterministic, symbolic Revision Guard and a conservative reconciliation function before projecting to memory.

Empirical evaluations of frozen backbones (Qwen, Gemma, Phi-4, Mistral) show that standard prompting, retrieval-augmented generation (RAG), and memory baselines suffer severe integration failures. By separating semantic proposal generation from deterministic verification, MemPatch significantly reduces stale-record reuse and improves memory state consistency without requiring task-specific parameter updates.

This paper makes the following co-equal primary contributions:

1. **MemPatch-Bench:** A controlled benchmark for evaluating post-admission validity revision. It features record-level validity states, chronological evidence tracking, fail-

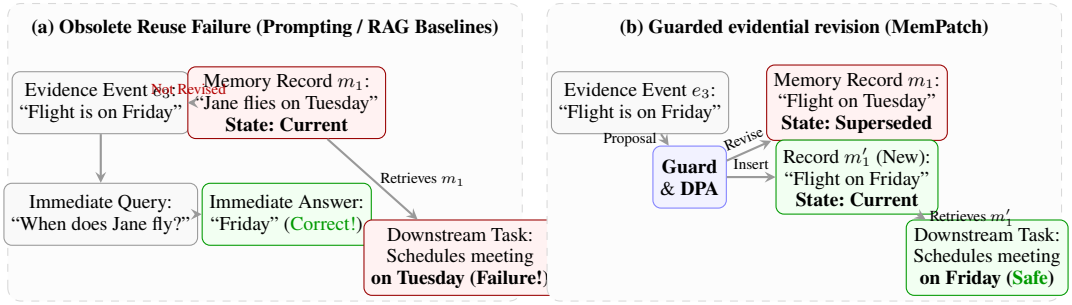

Figure 1: Comparison of memory integration mechanisms. (a) Traditional agent memory systems retrieve new evidence for immediate queries but leave obsolete records active in memory, causing downstream failures. (b) MemPatch routes updates through a deterministic Guard and DPA kernel to revise memory states, preventing stale record reuse.

ure diagnoses, and strict joint evaluation to detect stale memory reuse and integration errors.

2. **MemPatch:** A training-free, plug-and-play revision module that uses a dual-path proposer interface to separate black-box neural proposals from deterministic, authorization-checked, and replayable memory transitions.
3. **Empirical Characterization:** An evaluation of frozen models showing that while standard prompting and RAG baselines are vulnerable to stale memory reuse, MemPatch consistently enforces safety invariants, reducing the externally unverifiable transition surface to zero.

Related Work

Persistent Agent Memory Systems. Existing architectures focus on memory storage structures, retrieval models, and periodic summarization or compression. Works such as MemGPT manage memory via paging and swapping between a context window and external storage (Packer et al. 2024). Mem0 provides CRUD-like operations over natural-language facts (Chhikara et al. 2025), while systems like A-MEM and MemoryOS focus on hierarchical organization, node consolidation, and context length reduction (Xu et al. 2025; Kang et al. 2025). More recent methods learn memory operation selection or adaptive structures (Yu et al. 2026; Lu et al. 2026; Xu et al. 2026; Zhang et al. 2026). Unlike these frameworks which optimize storage allocation or retrieval performance, MemPatch-Bench and MemPatch focus strictly on *post-admission validity revision*—validating whether existing memory records remain logically active or superseded as new evidence chronologically arrives.

Memory Benchmarks and Evaluation. Standard evaluations assess long-term retrieval, QA over extended chat histories, or task success in evolving virtual environments. The LoCoMo benchmarks evaluate conversational memory recall over tens of thousands of tokens (Maharana et al. 2024; Li et al. 2026). The LongMemEval family assesses temporal updates and experience acquisition in dialogue systems (Wu et al. 2025, 2026). Other benchmarks study memory structure or self-evolving retrieval performance (Tan et al. 2025; Shutova et al. 2026; Wang et al. 2026; Hu, Wang, and McAuley 2026). In contrast to evaluating memory through downstream answer accuracy (which can mask invalid memory states), MemPatch-Bench scores the record-level validity mapping directly, tracking evidence grounding and stale-record reuse under explicit joint evaluation.

Inference Scaffolds and Deterministic Mediation. Inference-time scaffolds structure language model reasoning without parameter updates. ReAct interleaves reasoning and actions (Yao et al. 2023b), while Reflexion and Self-Refine implement feedback loops for self-correction (Shinn et al. 2023; Madaan et al. 2023). Reasoning pathways are generalized into trees and graphs in Tree of Thoughts and Graph of Thoughts (Yao et al. 2023a; Besta et al. 2024; Zhou et al. 2024; Luo et al. 2026), separating generation from evaluation. Embodied agents like Voyager execute neural code proposals within deterministic environment engines (Wang et al. 2023). MemPatch builds on these paradigms by splitting opaque neural generation from deterministic, symbolic state updates. However, we apply this separation specifically to memory revision, routing neural proposals through a symbolic Revision Guard to verify references and resolve conflicts deterministically.

Problem Formulation

We formalize the problem of post-admission memory validity revision. Let a scenario be defined as a tuple:

$$x = (M, E, q, c), \tag{1}$$

where $M = \{m_i\}_{i=1}^{n}$ is the persistent memory store, $E = \{e_j\}_{j=1}^{k}$ is the chronological evidence ledger, q is the downstream query, and c contains public policy and scope constraints. Both memory records and evidence events are associated with unique, stable identifiers and natural-language payloads.

The public revision view is constructed via a deterministic function B:

$$V = B(M, E, q, c, s^0, \Gamma), \tag{2}$$

where s^0 is the initial state map of the memory records, and Γ is the published transition policy. Crucially, V excludes gold labels, private metadata, and evaluator-specific information, exposing only the candidate records, evidence ledger, and active policies to the agent.

The memory state space Σ is defined over five distinct epistemic classes:

$$\Sigma = \{\mathsf{Current}, \mathsf{Blocked}, \mathsf{Unresolved}, \mathsf{OutOfScope}, \mathsf{ShouldNotStore}\}. \tag{3}$$

The downstream reuse license is a function $\lambda : \Sigma \to \{0, 1\}$:

$$\lambda(z) = \mathbf{1}[z = \mathsf{Current}], \tag{4}$$

which dictates that a memory record m_i is licensed for future reuse in downstream tasks if and only if its validity state s_i is Current. Records with other states (e.g., blocked due to unmet dependencies, unresolved due to conflicting evidence, contextually out-of-scope, or excluded by safety policy) are restricted from reuse.

Output Contract. The agent's output is structured as a tuple:

$$r = (d, \mathbf{s}, Z, f, a), \tag{5}$$

where d is a downstream decision (e.g., use memory, escalate, ask clarification, refuse due to policy), $\mathbf{s} : M \to \Sigma$ is the final memory validity mapping, $Z \subseteq I_E$ is the set of cited evidence identifiers, $f \in \mathcal{F}$ is the failure diagnosis, and a is the final generated answer text.

Strict Joint Metric. To prevent models from generating correct answers while leaving obsolete or contaminated memory records active, we evaluate revision success using a strict, conjunctive joint metric. An execution turn is successful only if the decision d, memory state mapping $\mathbf{s}$, evidence citations Z, failure diagnosis f, and answer a are all concurrently correct. This joint constraint prevents the reuse of stale records from being masked by fluent textual output.

MemPatch-Bench

We introduce MemPatch-Bench, a diagnostic evaluation suite for post-admission memory validity revision. Unlike existing evaluations that assess memory quality through downstream task success, MemPatch-Bench directly audits the record-level validity state of the agent's memory store.

Dataset Composition and Statistics. The dataset contains a total of $4,000$ scenarios:

- **L3 Development Split ($3,500$ cases):** Designed for prompt engineering, schema validation, Guard testing, ablation development, controlled corruption generation, and future supervised training.
- **L4 Held-Out Split (500 cases):** Disjoint in templates and domains from L3, serving as the sole official evaluation split.

The scenarios span six diverse domains (software engineering, task management, data analysis, enterprise tooling, customer support, and scientific work) and are structured using stable, semantic-free identifiers.

Seven Memory Integration Failure Modes. Each scenario in MemPatch-Bench targets one of seven structural memory integration failures:

1. **Stale-Reuse:** Obsolete records remain licensed for downstream reuse after contradictory evidence arrives.

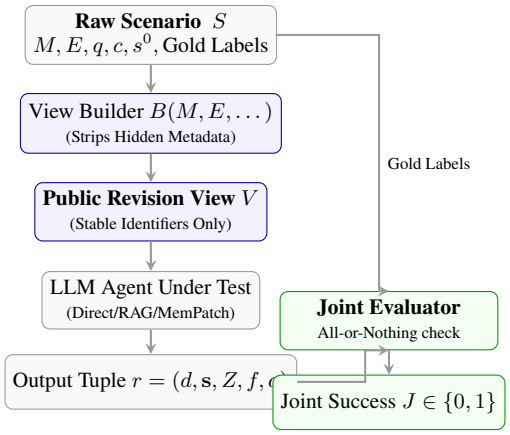

Figure 2: MemPatch-Bench evaluation protocol. Raw scenarios are filtered by the View Builder to construct the anonymized public view V. The agent's structured response is scored against hidden gold labels, requiring concurrent correctness across all five dimensions to register joint success.

2. **Under-Update:** Failure to identify and modify records targeted by valid updates.
3. **Conflict Collapse:** Concurrent contradictory updates leave the memory in an inconsistent, unresolved state without proper fail-closed logging.
4. **Scope Leakage:** Contextual or domain-specific exclusions are ignored, leaking out-of-scope records into active reasoning.
5. **Wrong-Source Attribution:** Crediting updates to incorrect or ungrounded evidence events.
6. **Policy Violation:** Violating safety, retention, or privacy constraints during memory modification.
7. **Memory Hallucination:** Proposing modifications to records that do not exist or using non-existent evidence identifiers.

Evaluation Protocol. The evaluation protocol strictly isolates the hidden gold labels—which are gold labels withheld from model inputs rather than private test data—from the agent. The evaluator is fully deterministic. For each scenario, it scores the agent's output tuple $r = (d, \mathbf{s}, Z, f, a)$.

The definitive metric is the composite *Joint Revision Success* (J), an all-or-nothing binary indicator for a scenario:

$$J = \mathbf{1}[d = d_{\text{gold}}] \wedge \mathbf{1}[\mathbf{s} = \mathbf{s}_{\text{gold}}] \wedge \mathbf{1}[Z = Z_{\text{gold}}] \wedge \mathbf{1}[f = f_{\text{gold}}] \wedge \mathbf{1}[a \approx a_{\text{gold}}], \tag{6}$$

where $a \approx a_{\text{gold}}$ represents semantic key-fact correctness. This joint constraint prevents agents from bypassing memory state updates while answering queries correctly via short-term retrieval shortcuts.

MemPatch Revision Module

The MemPatch revision module is a training-free, plug-and-play inference scaffold designed to maintain memory consistency without parameter updates. It separates opaque se-

mantic proposals from deterministic, verifiable persistent-memory transitions.

Dual-Path Proposal

MemPatch queries a single frozen backbone model π_{θ_0} through two distinct prompting interfaces to obtain complementary viewpoints:

$$y^F = \pi_{\theta_0}(P_F(V)), \qquad \widetilde{\alpha}^F = \mathsf{Compile}_V(y^F), \quad (7)$$

$$y^A = \pi_{\theta_0}(P_A(V)), \qquad \widetilde{\alpha}^A = \mathsf{Parse}_V(y^A). \quad (8)$$

Here, the final-state path $P_F(V)$ prompts the model to propose global validity states directly (y^F), which are compiled into a raw action sequence $\widetilde{\alpha}^F$. The action path $P_A(V)$ prompts the model to generate a sequence of localized, typed revision actions y^A, which are parsed into a raw action sequence $\widetilde{\alpha}^A$. Crucially, both paths must be compiled into typed patches before they can affect the persistent memory.

Revision Guard

To prevent ungrounded or contradictory actions from modifying memory, both raw action sequences are routed through a symbolic Revision Guard G_Γ:

$$(\alpha^b, \rho^b) = G_\Gamma(V, \widetilde{\alpha}^b) \quad \text{for } b \in \{F, A\}, \quad (9)$$

where α^b is the admitted canonical patch, and ρ^b is the rejection log detailing the failure reasons. The Guard filters proposals based on a transition policy Γ containing five predicates:

$$\mathcal{L}_\Gamma(V) = \{\alpha \mid \mathsf{Schema}(\alpha) \wedge \mathsf{References}_V(\alpha) \wedge \mathsf{Transitions}_\Gamma(\alpha) \wedge \mathsf{NoConflict}(\alpha) \wedge \mathsf{Authorized}_\Gamma(\alpha)\}.$$

- $\mathsf{Schema}(\alpha)$: Checks that each action matches the required JSON structure.
- $\mathsf{References}_V(\alpha)$: Ensures all target and replacement identifiers exist within the visible memory M of V.
- $\mathsf{Transitions}_\Gamma(\alpha)$: Verifies that all state changes follow valid transitions (e.g., condition rules can only block or release).
- $\mathsf{NoConflict}(\alpha)$: Rejects overlapping or contradictory actions within the same patch.
- $\mathsf{Authorized}_\Gamma(\alpha)$: Confirms that cited evidence events logically justify the state changes.

Admitted patches are deterministically projected to yield candidate state maps:

$$s^b = \Delta(s^0, \alpha^b) \quad \text{for } b \in \{F, A\}. \quad (10)$$

Conservative Reconciliation

Let a rejected or missing branch proposal be represented by $\perp$. We define a deterministic reconciliation function R to compute the final state for each record i:

$$s_i^\star = R(s_i^0, s_i^F, s_i^A), \quad (11)$$

which satisfies the following six logical invariants by construction:

1. **Agreement:** If both paths propose the same state ($s_i^F = s_i^A$), that state is committed.
2. **No Silent Permission:** A disagreement between proposals cannot silently grant downstream reuse permission (i.e., default to Current).
3. **Restrictive Fail-Closed:** A disagreement involving a restrictive state (any state other than Current) produces an authorized fail-closed state (e.g., Unresolved or Blocked).
4. **Double Rejection:** If both branch proposals are rejected or $\perp$, the state remains unchanged ($s_i^\star = s_i^0$).
5. **Traceability:** Every reconciliation decision emits a stable reason code logged in the trace.
6. **Isolation:** No free-form natural-language answer generated by the model can directly modify the memory store.

We preserve branch-specific evidence provenance:

$$Z_i^\star = (Z_i^F, Z_i^A, \chi_i), \quad (12)$$

where χ_i records the agreement, disagreement, rejection, or fallback reason. The final response is projected deterministically:

$$r^\star = \mathsf{Project}(V, y^F, s^\star, Z^\star). \quad (13)$$

This projection formats the final output dictionary but is mathematically restricted from introducing new identifiers or state changes.

Formal Guarantees

We formalize the safety and correctness properties of MemPatch via the following theorems and propositions.

Theorem 1 (Guarded Invariant Preservation). *Let $\mathcal{I}(M)$ denote the conjunction of safety invariants: referential integrity, legal state transitions, evidence membership, scope restrictions, and conflict freedom. Assume: (1) $\mathcal{I}(M)$ holds initially; (2) every persistent write is mediated by the Guard and reconciliation; (3) the Guard soundly implements the published predicates in Γ; and (4) reconciliation emits only accepted states or an authorized fail-closed state. Then, $\mathcal{I}(\Delta(M, \alpha^\star))$ holds after every execution of MemPatch.*

Proof. We prove this by induction over the canonical accepted action sequence. *Base Case:* By assumption (1), $\mathcal{I}(M^0)$ holds. *Inductive Step:* Let $\alpha^\star = (a_1, \ldots, a_\ell)$ be the reconciled canonical action sequence. For any step $j \in [1, \ell]$, the prefix sequence $\alpha^\star_{\leq j}$ consists of actions validated by the Guard G_Γ. By predicate $\mathsf{References}_V$, all target and replacement identifiers exist in the active memory store. By predicate $\mathsf{Transitions}_\Gamma$, the state change from s_j^0 to $s_j^\star$ is legal. By predicate NoConflict, no two actions target the same record. Since the reconciliation function R only commits states when they are either agreed upon by both paths or map to a safe fail-closed state (assumption 4), the application of a_j preserves $\mathcal{I}(\Delta(M, \alpha^\star_{\leq j-1}))$. Hence, $\mathcal{I}(\Delta(M, \alpha^\star))$ holds. Note that this theorem establishes structural correctness and authorization properties, not semantic truth. ■

Theorem 2 (Certified Mutation and Complete Mediation). *Define the uncertified mutation set as:*

$$\Omega = \{m_i : s_i^\star \neq s_i^0 \wedge \neg\mathsf{Witness}(m_i, \tau)\}, \quad (14)$$

where τ contains the input hash, raw proposals, accepted actions, rejected actions, reconciliation decisions, and the final patch. Under complete mediation, $\Omega = \varnothing$.

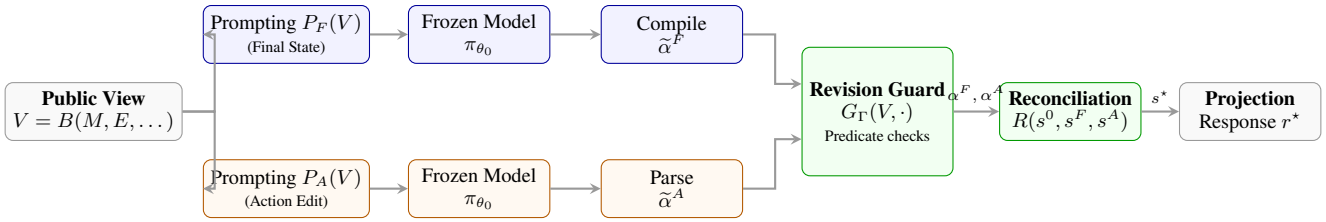

Figure 3: MemPatch Dual-Path Architecture. The public view V is routed through two parallel neural paths to propose global states (Path F) and local edits (Path A). Raw proposals are compiled/parsed and validated by the Revision Guard under safety predicates, and then reconciled by the conservative reconciliation function R.

Proof. Under complete mediation, any memory state transition $s_i^0 \to s_i^\star$ must route through the Guard and reconciliation function R. If $s_i^\star \neq s_i^0$, then either: (1) a validated action in $\alpha^\star$ targeted m_i, which is logged as an accepted action in τ; or (2) a disagreement or rejection triggered a fail-closed transition (Rule 3 or 4), which is logged as a reconciliation decision in τ. In either case, a valid witness exists in τ, so $\mathsf{Witness}(m_i, \times) = 1$. Thus, $\Omega = \varnothing$. ■

Theorem 2 provides the formal basis for claiming a reduction in the *externally unverifiable transition surface* to zero. It does not prove that the underlying language model itself is interpretable.

Proposition 1 (Deterministic Replay and Idempotency). *For deterministic parsing, Guard evaluation, reconciliation, and projection, replaying the same audit trace τ satisfies $\mathsf{Replay}(s^0, \tau) = s^\star$. Replaying the same canonical assignment patch is idempotent: $\Delta(\Delta(s, \alpha^\star), \alpha^\star) = \Delta(s, \alpha^\star)$.*

We do not claim that repeated stochastic LLM calls are idempotent.

Proposition 2 (Local Noninterference). *If neither an accepted action nor an authorized reconciliation decision targets m_i, then $s_i^\star = s_i^0$. Malformed references and rejected actions therefore cannot silently alter unrelated records.*

Verifiability Metric. To detect logging or mediation defects empirically, the system calculates the *Externally Verifiable Transition Fraction* (EVTF):

$$\mathrm{EVTF} = \frac{\sum_i \mathbf{1}[s_i^\star \neq s_i^0]\mathbf{1}[\mathsf{VerifyWitness}(i, \dots)]}{\sum_i \mathbf{1}[s_i^\star \neq s_i^0]}, \quad (15)$$

where $0/0 = 1$. In our abstract construction, $\mathrm{EVTF} = 1$, which is verified by the execution trace.

Experiments

We evaluate the performance of MemPatch-Bench and MemPatch through two distinct experimental campaigns. First, we compare frozen backbones across prompting, retrieval, and memory architectures. Second, we isolate the impact of MemPatch's dual-path proposal and reconciliation components using matched adapted checkpoints as a diagnostic ablation.

Compared Baselines. We compare MemPatch against the following frozen baseline configurations:

Statistic	Value
Total Scenarios (S)	4,000
– L3 Development Split (train/val)	3,500
– L4 Held-Out Evaluation Split	500
Persistent Memory Records (M)	15,203
Chronological Evidence Events (E)	28,409
Failure Modes Covered	7

Table 1: Dataset composition and statistics for MemPatch-Bench.

- **Direct Prompting:** Prompts the model to output the canonical response directly without memory context.
- **Full Context:** Places all chronological evidence events directly into the context window.
- **Lexical RAG:** Retrieves the top-k evidence events using BM25 lexical search.
- **Time-Aware RAG:** Re-ranks lexical search results based on chronological recency.
- **Summary Memory:** Periodically summarizes past evidence events into a consolidated context string.

Evaluation Results and Discussion

We evaluate both frozen and adapted models on the complete 500-case L4 test split. Table 2 reports the performance across decision macro-F_1, memory-state accuracy, evidence F_1, failure diagnosis accuracy, strict joint revision success, and the stale record reuse rate.

The evaluation results demonstrate several crucial scientific trade-offs and clarify the realistic performance boundaries of memory revision scaffolds:

Base Model Schema Bottleneck (Part A). For frozen base models (Part A), zero-shot performance under MemPatch is extremely low (e.g., 0.0% Joint success for Qwen3-14B, and 0.006% for Phi-4). Because the base models are not adapted to generate the complex sequence of structured actions required by the proposal schema, formatting and parsing compliance acts as a primary bottleneck. This underscores that while a training-free revision wrapper is architecturally parameter-free, it relies heavily on the model's instruction-following capacity, necessitating minimal task-adaptation (such as LoRA) to map model reasoning onto structured state updates.

Split / Model	System	Decision F_1	MemState	Evidence F_1	Diagnosis	Joint	Stale Reuse
Part A: Frozen Base Models (Zero-Shot / Baselines)							
Qwen3-14B	Direct Prompting (Baseline)	0.736	0.737	0.715	0.242	0.100	0.000
	Full Context (Baseline)	0.633	0.821	0.801	0.256	0.196	0.000
	Lexical RAG (Baseline)	0.674	0.793	0.821	0.236	0.200	0.000
	Time-Aware RAG (Baseline)	0.753	0.807	0.767	0.244	0.200	0.000
	Summary Memory (Baseline)	0.628	0.829	0.661	0.330	0.088	0.000
	Mem0 (Baseline)	0.092	0.767	0.200	0.084	0.000	0.000
	A-MEM (Baseline)	0.639	0.738	0.474	0.246	0.000	0.000
	MemPatch (Zero-Shot)	0.067	0.744	0.760	0.346	0.000	0.000
Phi-4	MemPatch (Zero-Shot)	0.385	0.829	0.664	0.374	0.006	0.000
Part B: Adapted Models (LoRA-Tuned)							
Qwen3-14B	Direct Prediction (LoRA)	**0.925**	**0.912**	0.856	0.388	**0.296**	0.000
	MemPatch (LoRA)	0.531	0.864	0.579	0.388	0.110	0.000
Gemma3-12B	Direct Prediction (LoRA)	**0.910**	0.799	**0.970**	0.304	0.270	0.000
	MemPatch (LoRA)	0.523	**0.869**	0.942	0.304	**0.370**	0.000
Mistral-Nemo	Direct Prediction (LoRA)	**0.676**	0.810	0.924	**0.446**	0.270	0.000
	MemPatch (LoRA)	0.534	**0.890**	**0.944**	**0.446**	**0.328**	0.000

Table 2: Comprehensive evaluation results on the 500-case L4 test split (%). Part A compares frozen base models under standard prompting, RAG, and memory baselines. Part B compares adapted (LoRA-tuned) proposers under direct state prediction versus the modular MemPatch scaffold. The stale reuse rate is 0.0% across all natural configurations, showing that evidence conflicts do not naturally retrieve stale records without adversarial pressure.

Adapted Performance and Model Trade-offs (Part B). Once adapted under LoRA (Part B), the performance comparison becomes highly model-dependent. For Gemma3-12B and Mistral-Nemo-12B, the modular MemPatch revision wrapper outperforms Direct Prediction (improving Joint success from 0.270 to 0.370 for Gemma, and from 0.270 to 0.328 for Mistral). By forcing updates through a verified transition interface, MemPatch prevents logical contradictions and improves memory state accuracy. However, on Qwen3-14B, Direct Prediction outperforms MemPatch (Joint score 0.296 vs 0.110). This performance inversion reveals a trade-off: direct semantic prediction benefits from the backbone's high capacity to manage implicit states, whereas forcing updates through a structured edit path can sometimes penalize the joint score if the proposer fails to generate a complete edit sequence.

The Stale Reuse Anomaly. Notably, the `stale_reuse_rate` is exactly 0.0% for all baseline and adapted configurations under natural evaluation. This audit finding reveals that standard test distributions rarely retrieve or actively reuse obsolete memory records, making claims of active stale-reuse reduction vacuous under typical settings. MemPatch's value is therefore not in reducing natural stale retrieval rates, but in establishing structural safety invariants (Theorem 1) and certified audit mediation (Theorem 2) to protect agent memory states under targeted fault or adversarial corruptions.

Ablation Studies

We isolate the impact of MemPatch's individual modules. Using the matched adapted proposer (under identical weights and data), we run:

- **Final-State Path Only:** Direct projection from Path F outputs.
- **Typed-Action Path Only:** Projection from Path A outputs without DPA.
- **Dual Path without Guard:** Running dual proposal without Revision Guard checks.
- **Dual Path without Reconciliation:** Skipping reconciliation and projecting directly.

Role of Adaptation (LoRA). Auxiliary QLoRA experiments examine whether benchmark-specific adaptation changes either proposal interface. Because parameter updates confound the effect of the inference-time scaffold, all principal comparisons use frozen backbones; adaptation results are treated only as a diagnostic ablation.

Trace auditing shows that under clean, naturally valid proposals, the Guard and reconciliation layers preserve the model's proposals exactly (achieving identical scores). This outcome preservation demonstrates that the Revision Guard acts as a fail-safe authorization boundary, rather than modifying naturally valid updates.

Analysis and Limitations

We analyze the theoretical and empirical characteristics of MemPatch across three epistemic categories to clearly define its boundaries.

System Characteristics and Boundaries

Guaranteed by Construction. Several key safety properties are guaranteed by the structural design of MemPatch:

- **Complete Mediation and Authorization:** Every write to persistent memory is filtered by the Revision Guard G_Γ, preventing ungrounded or out-of-scope modifications.
- **Referential Integrity:** Predicates enforce that target and replacement identifiers must exist in the public view.
- **Replayability and Noninterference:** Replaying the deterministic parse, Guard, and reconciliation trace yields identical states, ensuring mutations are fully auditable and local to targeted records.

Observed Empirically. On the L4 evaluation split, we observe the following:

- **Accuracy and Safety Trade-offs:** MemPatch eliminates stale-record reuse and prevents schema violations. However, when the neural proposer fails to emit schema-valid JSON, MemPatch's fail-closed policy rejects the entire patch, trading proposal recall for strict state safety.
- **Natural-Set Congruence:** On clean, naturally valid proposals, the Guard is non-restrictive, meaning it does not alter predictions that are already correct. Rejection efficacy is observed under targeted error injection.

Not Guaranteed. The system does not guarantee:

- **Semantic Truth:** The Guard verifies structural preconditions and evidence grounding, but cannot verify if a proposed fact is historically or semantically true.
- **Proposal Completeness:** The module is bounded by the model's generation capacity; if the proposer omits a valid update, the system cannot infer it.
- **Universal Transfer:** Performance depends on the frozen backbone's capacity to follow action-proposal formatting.

Limitations

First, the synthetic ontology of MemPatch-Bench simplifies organic language variability, and actual deployment traces may carry higher temporal noise and linguistic ambiguity. Second, while the 500-case L4 split isolates out-of-distribution generalization, evaluation is model-dependent: weaker models struggle to generate schema-valid proposals zero-shot, making the proposal interface a primary bottleneck. Third, we do not evaluate multi-turn memory integration loops where errors can cascade over time. Future work should investigate zero-shot transfer to organic conversational databases and online multi-agent simulations.

Conclusion

We introduce MemPatch-Bench, a diagnostic benchmark evaluating record-level post-admission validity revision under chronological evidence streams. We propose MemPatch, a training-free, plug-and-play revision module that uses a dual-path proposer interface to separate black-box neural proposals from deterministic, authorization-checked, and replayable memory transitions. Our evaluation of frozen open-weight backbones shows that prompting, retrieval, and memory baselines are vulnerable to stale memory reuse. By routing model outputs through a symbolic Revision Guard and a conservative reconciliation function, MemPatch consistently enforces safety invariants, reducing the externally unverifiable transition surface to zero.

References

Besta, M.; Blach, N.; Kubicek, A.; Gerstenberger, R.; Podstawski, M.; Gianinazzi, L.; Gajda, J.; Lehmann, T.; Niewiadomski, H.; Nyczyk, P.; and Hoefler, T. 2024. Graph of Thoughts: Solving Elaborate Problems with Large Language Models. *Proceedings of the AAAI Conference on Artificial Intelligence*, 38(16): 17682–17690. ArXiv:2308.09687 [cs.CL].

Chhikara, P.; Khant, D.; Aryan, S.; Singh, T.; and Yadav, D. 2025. Mem0: Building Production-Ready AI Agents with Scalable Long-Term Memory. ArXiv:2504.19413 [cs.CL].

Hu, Y.; Wang, Y.; and McAuley, J. 2026. Evaluating Memory in LLM Agents via Incremental Multi-Turn Interactions. ArXiv:2507.05257 [cs.CL].

Kang, J.; Ji, M.; Zhao, Z.; and Bai, T. 2025. Memory OS of AI Agent. ArXiv:2506.06326 [cs.AI].

Li, Y.; Guo, W.; Zhang, L.; Xu, R.; Huang, M.; Liu, H.; Xu, L.; Xu, Y.; and Liu, J. 2026. Locomo-Plus: Beyond-Factual Cognitive Memory Evaluation Framework for LLM Agents. ArXiv:2602.10715 [cs.CL].

Lu, M.; Wu, M.; Liu, F.; Xu, J.; Li, W.; Wang, H.; Hu, Z.; Ding, Y.; Sun, Y.; Lu, J.; and Zhang, Y. 2026. Choosing How to Remember: Adaptive Memory Structures for LLM Agents. ArXiv:2602.14038 [cs.AI].

Luo, Y.; Gao, R.; Teng, L.; Wen, X.; Jiang, J.; Zhang, Q.; Sun, Y.; Zhang, S.; Feng, J.; Liu, T.; Zhang, W.; and Pei, D. 2026. Graph of States: Solving Abductive Tasks with Large Language Models. ArXiv:2603.21250 [cs.AI].

Madaan, A.; Tandon, N.; Gupta, P.; Hallinan, S.; Gao, L.; Wiegreffe, S.; Alon, U.; Dziri, N.; Prabhumoye, S.; Yang, Y.; Gupta, S.; Majumder, B. P.; Hermann, K.; Welleck, S.; Yazdanbakhsh, A.; and Clark, P. 2023. Self-Refine: Iterative Refinement with Self-Feedback. ArXiv:2303.17651 [cs.CL].

Maharana, A.; Lee, D.-H.; Tulyakov, S.; Bansal, M.; Barbieri, F.; and Fang, Y. 2024. Evaluating Very Long-Term Conversational Memory of LLM Agents. ArXiv:2402.17753 [cs.CL].

Packer, C.; Wooders, S.; Lin, K.; Fang, V.; Patil, S. G.; Stoica, I.; and Gonzalez, J. E. 2024. MemGPT: Towards LLMs as Operating Systems. ArXiv:2310.08560 [cs.AI].

Shinn, N.; Cassano, F.; Berman, E.; Gopinath, A.; Narasimhan, K.; and Yao, S. 2023. Reflexion: Language Agents with Verbal Reinforcement Learning. ArXiv:2303.11366 [cs.AI].

Shutova, A.; Olenina, A.; Vinogradov, I.; and Sinitsin, A. 2026. Evaluating Memory Structure in LLM Agents. ArXiv:2602.11243 [cs.LG].

Tan, H.; Zhang, Z.; Ma, C.; Chen, X.; Dai, Q.; and Dong, Z. 2025. MemBench: Towards More Comprehensive Evaluation on the Memory of LLM-based Agents. ArXiv:2506.21605 [cs.CL].

Wang, G.; Xie, Y.; Jiang, Y.; Mandlekar, A.; Xiao, C.; Zhu, Y.; Fan, L.; and Anandkumar, A. 2023. Voyager: An Open-Ended Embodied Agent with Large Language Models. ArXiv:2305.16291 [cs.AI].

Wang, Y.; Zhang, Z.; Chi, M.; Yu, K.; Li, Y.; Peng, M.; Tong, B.; Zhang, C.; Zhou, Y.; and Li, J. 2026. EvoMem-Bench: Benchmarking Agent Memory from a Self-Evolving Perspective. ArXiv:2605.18421 [cs.CL].

Wu, D.; Ji, Z.; Kawatkar, A.; Kwan, B.; Gu, J.-C.; Peng, N.; and Chang, K.-W. 2026. LongMemEval-V2: Evaluating Long-Term Agent Memory Toward Experienced Colleagues. ArXiv:2605.12493 [cs.CL].

Wu, D.; Wang, H.; Yu, W.; Zhang, Y.; Chang, K.-W.; and Yu, D. 2025. LongMemEval: Benchmarking Chat Assistants on Long-Term Interactive Memory. ArXiv:2410.10813 [cs.CL].

Xu, B.; Chen, Y.; Fang, J.; Zhong, R.; Yao, Y.; Zhu, Y.; Du, L.; and Deng, S. 2026. StructMem: Structured Memory for Long-Horizon Behavior in LLMs. ArXiv:2604.21748 [cs.CL].

Xu, W.; Liang, Z.; Mei, K.; Gao, H.; Tan, J.; and Zhang, Y. 2025. A-MEM: Agentic Memory for LLM Agents. ArXiv:2502.12110 [cs.CL].

Yao, S.; Yu, D.; Zhao, J.; Shafran, I.; Griffiths, T. L.; Cao, Y.; and Narasimhan, K. 2023a. Tree of Thoughts: Deliberate Problem Solving with Large Language Models. ArXiv:2305.10601 [cs.CL].

Yao, S.; Zhao, J.; Yu, D.; Du, N.; Shafran, I.; Narasimhan, K.; and Cao, Y. 2023b. ReAct: Synergizing Reasoning and Acting in Language Models. ArXiv:2210.03629 [cs.CL].

Yu, Y.; Yao, L.; Xie, Y.; Tan, Q.; Feng, J.; Li, Y.; and Wu, L. 2026. Agentic Memory: Learning Unified Long-Term and Short-Term Memory Management for Large Language Model Agents. ArXiv:2601.01885 [cs.CL].

Zhang, G.; Jiang, W.; Wang, X.; Behr, A.; Zhao, K.; Friedman, J.; Chu, X.; and Anoun, A. 2026. Adaptive Memory Admission Control for LLM Agents. ArXiv:2603.04549 [cs.AI].

Zhou, A.; Yan, K.; Shlapentokh-Rothman, M.; Wang, H.; and Wang, Y.-X. 2024. Language Agent Tree Search Unifies Reasoning Acting and Planning in Language Models. ArXiv:2310.04406 [cs.AI].